Level-3 order-book replay on the raw market-data store

What we built on 23 October 2026 · C++23, GCC 14.2, AVX-512

Today we derived the Level-3 counterpart of the raw L1 store built in earlier sessions. Line handlers keep every order message (add, execute, cancel, delete, replace, modify) in raw form, routed to its symbol's append-only queue by a locate table (ITCH/TSX-style) or an order-id route table (PITCH-style). A TCP client names symbols, a feed-time start and a depth N (1–64; 20 as specified). The server rebuilds each symbol's aggregate book from the session open, sends a top-N snapshot as of the start, then the top N after every message that changes it, in feed-time order across symbols, and carries on live.

A review of that first version found three correctness bugs, two of them also in the L1 session, and a latent race in the shared symbol table. All are fixed, and the L1 and L3 read sides now share one session core. The tests pass with RAM and file backing, under TSan and ASan/UBSan, and replay before the start time is about twice as fast.

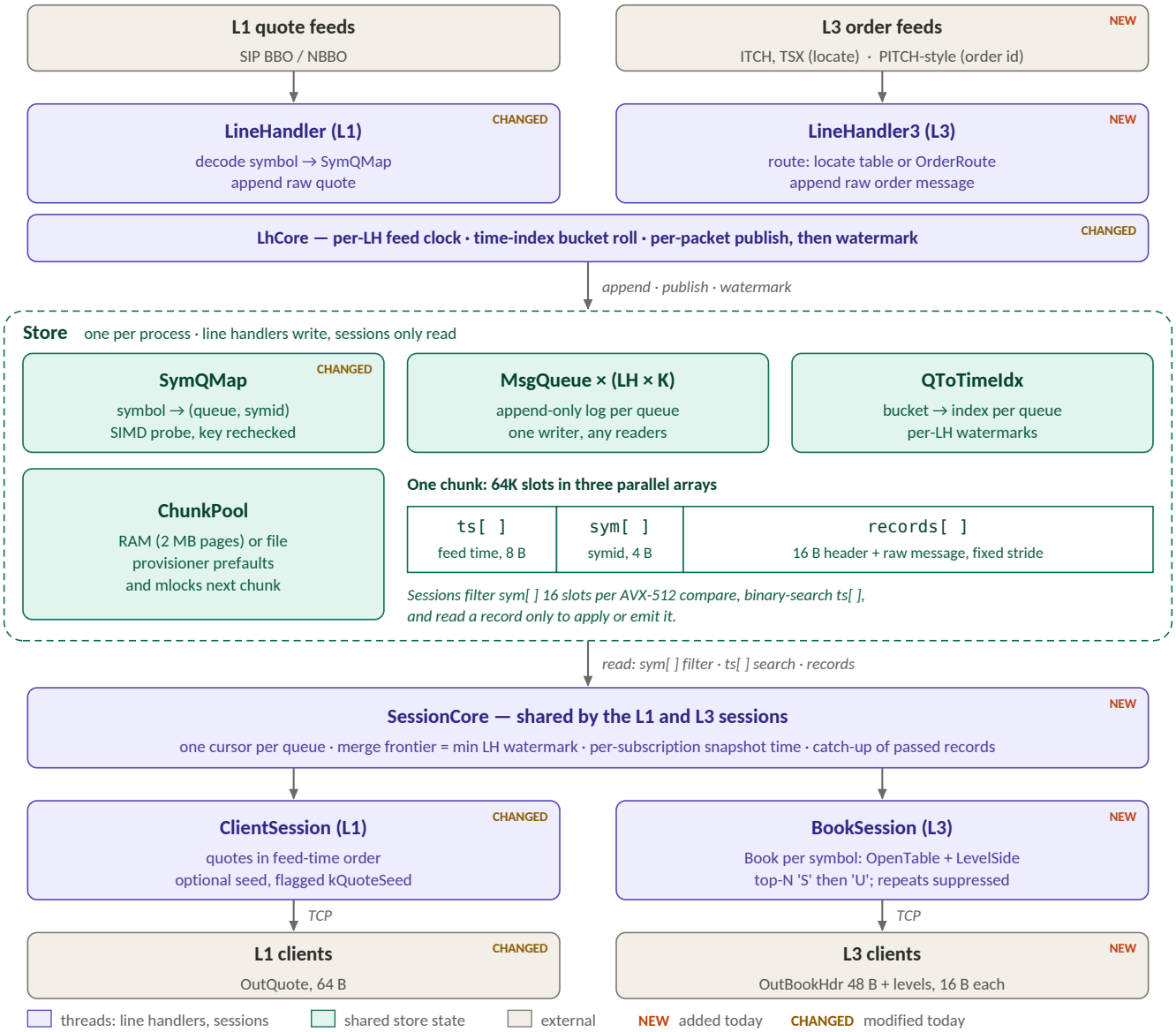


Figure 1. Components and data flow. Line handlers, one per pinned core, are the only writers; sessions only read, so a slow client never slows ingest. The L1 and L3 paths share the store and the session core; they differ only in routing on the way in and per-symbol state on the way out.

Components

Component	Role	Today
`LineHandler3`	Routes each L3 message to its symbol's queue: a 64K-entry locate table filled from directory messages, or OrderRoute for feeds that carry the symbol only on the add.	New
`OrderRoute`	Order id → (queue, symbol, remaining qty) on the line handler. 16-byte entries, four per cache line, on prefaulted 2 MB pages where available: one line per lookup in the common case.	New
`LhCore`	Per-LH feed clock (kept monotone), time-index bucket roll, per-packet publish, then the watermark. Shared by both line handlers.	Changed
`SymQMap`	Symbol → (global queue, symid). Lock-free reads with one AVX-512 compare per four keys; the key is now rechecked after the acquire.	Changed
`MsgQueue, ChunkPool`	Append-only log per queue; chunks of three parallel arrays (feed time, symid, fixed-stride raw records); RAM or file backing with a provisioner thread.	Unchanged
`QToTimeIdx`	Per-queue index at each time bucket, plus each line handler's watermark (latest published feed time).	Unchanged
`SessionCore`	Read-side core for both sessions: a cursor per queue, the merge frontier (min watermark, stale feeds excluded), per-subscription snapshot times, catch-up, bounded work steps.	New
`BookSession`	One Book per symbol: OpenTable of orders plus a LevelSide ladder per side (64-level SoA blocks). Publishes 'S' then 'U' records; repeats of the same top N are suppressed.	New
`ClientSession`	L1 quotes in feed-time order, with an optional flagged seed (latest quote before the subscription's start).	Changed

Snapshot timing

Each subscription has its own snapshot time. For a symbol known when the client connects it is START: the session applies every earlier message without sending it, sends 'S' (the book before START), then a 'U' for each later message that changes the top N. A symbol can also first appear after the client connects. That is common with order-id routing, where a symbol enters the table on its first add. It gets max(START, F + 1), where F is the latest feed time the session has consumed. Records its queue's cursor has already passed are replayed into its book first, and its 'S' goes out when the merge reaches that time. Output to the client never goes back in time, and 'S' plus the later 'U' records is complete for every symbol. This replaces a single session-wide 'live' flag and adds the catch-up; together they fix bugs 2 and 3 below.

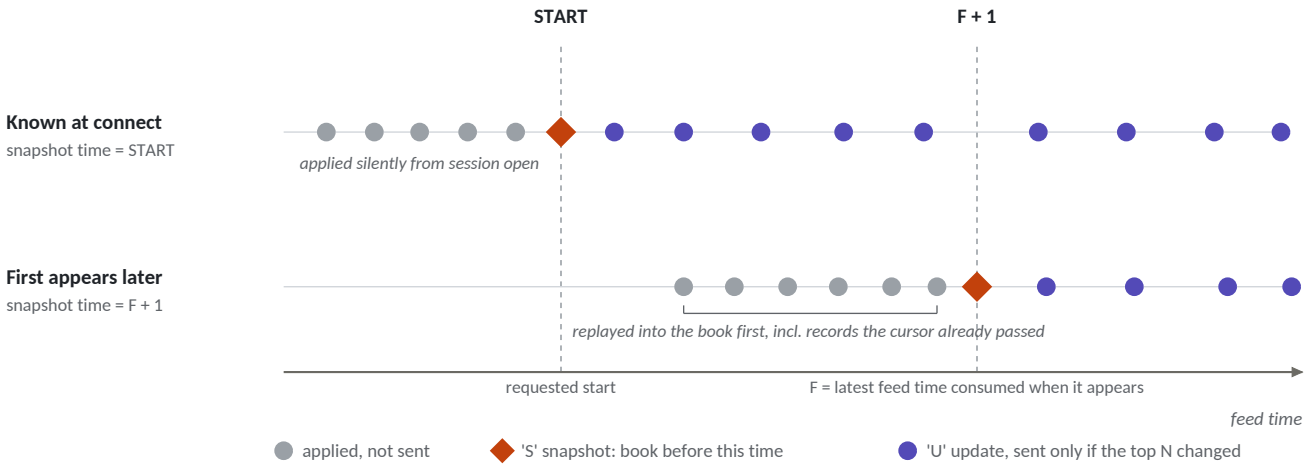


Figure 2. Two subscriptions in one session. The second symbol's records before F + 1 are applied but not sent; its snapshot carries their effect.

Client protocol (L3)

Request, one line: `<START_FEED_NS> <DEPTH 1..64> <SYM1> [SYM2 ...]`, up to 256 symbols. Response: a stream of `OutBookHdr` records (48 bytes), each followed by `nbid` then `nask` levels of 16 bytes (price, quantity, order count).

Field	Meaning
`kind`	'S' snapshot: the book after every message before feed_ns. 'U' update: sent after a message that changed the top N on either side.
`feed_ns`	'S': the snapshot time (START, or F + 1 for a late symbol). 'U': the message's feed time. Non-decreasing across the stream.
`seq, trigger, rx_ns`	Per-symbol update sequence (0 for 'S'); the feed message type that caused a 'U'; its receive time.
`px_exp`	Price scale of the feed: price = integer × 10^{px_exp}.
`flags`	`kFlagInconsistent`: the book has seen an unknown or duplicate order id, e.g. the store started after the open without a snapshot.

L1 is unchanged on the wire except for a flags byte in `OutQuote`; seeds now carry `kQuoteSeed`, since a symbol that appears later can receive its seed mid-stream.

Review: what was wrong and how it was fixed

Each bug was reproduced against the first version before it was fixed; the extended tests fail if either session fix is reverted.

Bug	Effect	How confirmed	Fix
1. The order table lost live orders: `insert()` could write up to 7 slots past where `erase()` and `grow()` stop scanning.	Phantom liquidity left in a level on the read side; dropped messages in order-id routing.	Repro: 5 of 68 keys unfindable after 2 deletes; 5 of 70 dropped by one grow.	Probes walk aligned 64-byte lines; a zero sentinel bounds deletes; grow rehashes every slot; duplicate ids are detected.
2. A symbol not yet known at connect lost its first messages when its queue's cursor had already passed them. L3 and L1.	Wrong book for the rest of the day; gaps in L1 output.	Repro: symbol BBB lost its first two orders.	Per-subscription snapshot time; records the cursor passed are replayed into the book.
3. A session with no resolved symbols went live at once: with no feed to wait on, its frontier was unbounded.	Empty snapshot stamped START, then updates timestamped before START.	Repro: updates at +101 and +102 ns with START at +150 ns.	No session-wide live flag; each subscription snapshots when the merge reaches its time.
4. Torn read in `SymQMap::find`: a 64-byte load is not architecturally atomic.	A half-written key could match a shorter symbol and misroute it. Latent.	Found in review; TSan reports the read once symbols arrive mid-session.	Key rechecked after the acquire, one 16-byte compare; `tsan.supp` covers the intentional read.

Also from the review:

- A 'U' is sent only if the top N differs from the last one sent, so updates are exact rather than conservative.
- Books flag inconsistent input (`kFlagInconsistent`) instead of failing silently.
- OrderRoute rebuilt for one cache line per lookup; duplicate order ids counted on both sides.
- Sessions work in bounded steps, so stop requests and client disconnects are noticed during a long replay.
- Feed staleness is counted from when a session starts reading a line handler, not from time zero.

Verification

Suite	What it checks	Result
Unit tests	OpenTable: a cluster that fills the tail padding, deletes at its front, then overflows into a grow; 3M random operations against std::unordered_map. OrderRoute: the same at fixed size (a full tail refuses, never corrupts). LevelSide: 200K random adds and removals per side against std::map, rank checked on every operation. Book: duplicate and unknown ids.	All pass
L3 end-to-end, locate and order-id routing	2 line handlers, 400 symbols, ITCH-shaped flow with adds, deletes, cancels, executions, modifies and replaces. 16 symbols appear 2 s in, after the client connects; one requested symbol never appears. Every snapshot and update is compared with a reference book.	19/19 snapshots and every update (5,200–5,450 per run) match; output in feed-time order; no flags
L1 end-to-end	Record by record against the producers' quotes, including the seed and unbroken run for each late symbol.	All match
Mutation checks	Re-break each session fix: disable the catch-up; pin every snapshot to START.	Both suites fail: ~800 mismatched updates with flags set; out-of-order output
Sanitizers and backing	TSan (one documented suppression), ASan/UBSan; RAM and file-backed stores.	All pass

Performance

Same 2-vCPU cloud VM for both columns; absolute numbers will be lower on an isolated core, the ratios are what matter.

Measurement	First L3 version	After review
Replay before START: 6M messages, 20 books, 16 queues	124–139 ms	62–71 ms
Order-id routing on the line handler: cold 4M-slot table, 2M live orders	94–100 ns/msg	74–77 ns/msg
Book apply: 100K live orders, 2,000 levels per side (best of 5)	111–127 ns/event	106–127 ns/event

Replay got faster because each step reads the commit index once per queue, and a queue whose subscriptions are all still before their snapshot is drained without the cross-queue merge. The Book first came out 3–15% slower after the table fix; that was inlining around `apply()`, now fixed.

Still open

- **Starting mid-day.** Books are incomplete without a snapshot source (Glimpse or a spin server). The inconsistency flag fires only once a message references an order the book never saw.
- **ITCH timestamps** are nanoseconds since midnight. The real adapter must add the session date so client start times and the time index share one clock.
- **Feed-wide clear events** have no route to every symbol's book.
- **Replay cost for a late symbol.** Its catch-up scans the queue from index 0, worst case ~40 ms per 100M records. Recording each symbol's first index in the line handler would make it exact.
- **Same-timestamp sweeps** produce one update per execution. Coalescing by symbol and timestamp would cut bandwidth; it is a semantic choice.
- **Very deep books.** The blocked ladder costs ~25 ns per operation at 200 levels and ~150 ns at 20,000. A dense tick-indexed ladder would make lookup O(1) for those names.
- **File backing writeback**, from the earlier review, is unchanged.

Files

File	Lines	Contents	Today
`rawmd.hpp`	715	Store core: symbol key, SymQMap, ChunkPool, chunk layout, MsgQueue, QToTimeIdx, Store, LhCore, L1 LineHandler, QueueCursor	**Changed**
`server.hpp`	384	Output buffer, SessionCore, L1 ClientSession, Server	**Changed**
`l3.hpp`	603	L3 protocol contract, OpenTable, OrderRoute, LevelSide, Book, LineHandler3, OutBookHdr	**New**
`l3_server.hpp`	133	L3 request parsing, BookSession	**New**
`toy_l3.hpp`	75	Stand-in ITCH-shaped wire format, plus an order-id-routed variant	**New**
`toy_l1.hpp`	57	Stand-in L1 wire format	**Changed**
`l3_tests.hpp`	164	Unit tests for the L3 containers	**New**
`main_l3.cpp`	341	L3 unit and end-to-end tests, Book benchmark	**New**
`main.cpp`	192	L1 end-to-end test	**Changed**
`tsan.supp`	4	TSan suppression for the intentional SymQMap read	**New**

Build and run

```
g++ -std=c++23 -O3 -march=sapphirerapids -pthread main_l3.cpp -o rawmd_l3 && ./rawmd_l3
g++ -std=c++23 -O3 -march=sapphirerapids -pthread main.cpp -o rawmd && ./rawmd
./rawmd_l3 /path/on/nvme/pool.bin        # file-backed store
g++ -std=c++23 -O1 -g -fsanitize=thread -march=sapphirerapids -pthread main_l3.cpp -o rawmd_l3_tsan
TSAN_OPTIONS=suppressions=tsan.supp ./rawmd_l3_tsan
```